

The Hidden Cost of Delegation: The Economics of Complexity Ownership

Software engineering has a persistent piece of advice:

Don't reinvent the wheel.

The advice is usually correct.

If someone has already solved a problem, using the existing solution can save substantial implementation effort. Libraries, frameworks, platforms, managed services, and now AI agents all exist partly to let us delegate work instead of doing everything ourselves.

But there is a problem with how we usually evaluate delegation.

We measure the work that disappears.

We rarely measure the complexity that moves.

A tool may remove thousands of lines of code while introducing a new mental model, operational burden, architectural dependency, or migration cost.

A framework may make an application dramatically cleaner while requiring the entire team to think in its abstractions.

An advanced programming model may make concurrency elegant while turning a local choice into a system-wide commitment.

An AI agent may implement an entire feature in minutes while creating a new verification problem.

None of these observations imply that delegation is bad.

They imply something more important:

Delegation is an architectural and economic decision about where complexity will live and who will own it.

Once this becomes the starting point, many seemingly unrelated engineering decisions begin to look like instances of the same problem.

The wrong question

A common engineering decision looks like this:

Should we build this ourselves or use an existing tool?

Usually the comparison is presented as:

```
Build:
    2 weeks

Use tool:
    2 days
```

The tool wins.

But the real comparison is closer to:

```
Build:
    implementation
    testing
    maintenance

Use tool:
    learning
    integration
    configuration
    operational knowledge
    debugging
    upgrades
    limitations
    workarounds
    dependency management
    migration cost
```

The second list is often invisible in the initial proposal because much of it does not look like coding.

It is still engineering work.

The important question is therefore not:

"How much code does this save?"

It is:

"What is the total engineering cost of owning this decision?"

Complexity Does Not Disappear

A useful way to think about this is that complexity is rarely eliminated by delegation.

It is relocated.

Suppose we implement a capability ourselves.

```
Our application
  ↓
Our code
  ↓
Our complexity
```

Now suppose we use a library.

```
Our application
  ↓
Our code
  ↓
Library
```

Some implementation complexity has moved into the dependency, while the application still controls the overall behavior.

Now consider a framework.

```
Application
  ↓
Framework programming model
  ↓
Framework runtime
```

More responsibility has moved into the framework's model.

Finally, consider an external execution platform:

```
Application
  ↓
External platform
  ↓
Platform-defined execution
```

At this point, part of the architecture itself may have moved outside the application.

The reduction in application code is real.

So is the increase in dependency.

This is why "less code" and "less complexity" are not equivalent.

The Abstraction Boundary Matters

There is a fundamental difference between **using a capability** and **building inside an execution model**.

A library can provide a capability while allowing the application to retain ownership of its flow.

The application decides:

- how components interact,
- where policies live,
- how state is managed,
- what constitutes success,
- how failures are handled,
- and how the pieces are composed.

The library supplies a mechanism.

An external platform can be different.

A data-processing or integration platform may define the topology, scheduling, state, queues, retries, configuration model, deployment model, and operational behavior through its own abstractions.

At that point, the application is no longer simply using the platform.

Part of the application is being **expressed inside the platform**.

That distinction is easy to miss because both situations may be described with the same sentence:

"We are using an existing tool."

But architecturally they are very different.

One gives the application a capability.

The other can become the owner of an execution model.

Open Source Does Not Automatically Mean Ownership

There is another subtle trap.

An external platform being open source does not mean that the application owns the resulting architecture.

You may be able to inspect the platform's source code.

You may even be able to fork it.

But if your application's architecture depends on:

- its runtime,
- its configuration model,
- its workflow semantics,
- its internal state model,
- and its operational assumptions,

then the architecture is still coupled to that platform.

Source availability and architectural ownership are different properties.

Having access to the source code of a system does not necessarily give you control over how your application is structured around it.

The Cost of a Mental Model

Perhaps the least visible cost of a technology is the mental model it introduces.

Some dependencies are shallow.

A utility library solves a narrow problem.

You learn its API and continue thinking primarily in the application's own concepts.

Other technologies are much deeper.

They introduce a way of building systems.

Functional effect systems, reactive programming models, actor systems, workflow engines, and stream-processing runtimes can be exceptionally powerful.

They can also be exceptionally demanding.

The problem is not that they are complicated.

The problem is that their **conceptual model becomes part of your application**.

Once a programming model is adopted deeply enough, you may reach a point where most of the code must be expressed according to its rules.

A local technical decision has become a global architectural commitment.

This is why a sophisticated library can sometimes feel heavier than an ordinary application framework.

The question is not simply:

"Is this a library?"

The better question is:

"How much of my codebase has to think in this library's model?"

Local Abstraction vs. Global Programming Model

This creates an important distinction.

A local abstraction might look like:

```
Application
├─ domain
├─ application logic
├─ infrastructure
│   ├─ library A
│   ├─ library B
│   └─ library C
```

A global programming model looks more like:

```
Application
└─ programming model
    ├─ component A
    ├─ component B
    ├─ component C
    └─ almost everything else
```

The second model is not inherently bad.

In fact, sometimes it is exactly what you need.

But it deserves a different threshold for adoption.

A useful question is:

Does the problem require this programming model, or does the programming model merely make the implementation more elegant?

That distinction matters.

Elegance Can Be Expensive

Functional and reactive approaches illustrate this particularly well.

They can produce remarkably clean code.

Composition can become clearer.

Error handling can become explicit.

Resource management can become disciplined.

Concurrency can become more principled.

These are real benefits.

The danger is not that the abstraction is fake.

The danger is that **local elegance can hide global commitment.**

A programming model can make one problem beautiful while making every other part of the system conform to it.

The right comparison is therefore not:

"Which code looks cleaner?"

It is:

"Does the reduction in complexity justify the amount of the system that must adopt the model?"

A Personal Lesson: The Dark Side of "The Right Tool"

One of the most instructive kinds of engineering experience is the technology that looked like exactly the right answer.

There was a tool for the problem.

It promised to eliminate implementation.

It seemed like the responsible engineering choice.

And then the implementation became a prolonged struggle with the tool's model.

Understanding the system took time.

Understanding its boundaries took more time.

Working around its limitations took more.

Eventually, the experience can leave an engineer wondering:

"Why didn't I just write the thing?"

The answer is usually not that the tool was bad.

The tool may have been perfectly good at solving the general problem it was designed to solve.

The mismatch was economic and architectural.

The problem was more specific than the tool's abstraction.

The tool was solving a broader problem, and the application had to pay for that generality.

This is one of the reasons experienced engineers sometimes become suspicious of seemingly convenient abstractions.

Not because they oppose reuse.

Because they have learned that **adoption itself has a cost**.

The Architecture Tax

General-purpose tools have a characteristic cost that is easy to overlook.

They solve a broader problem than yours.

Suppose the actual requirement is:

A → B

A platform might provide:

```
A
↓
routing
↓
transformation
↓
```

```
scheduling
↓
state management
↓
retries
↓
backpressure
↓
parallelism
↓
observability
↓
B
```

That breadth is valuable when you need it.

But every concept is also part of the cognitive surface area you must understand.

This creates an **architecture tax**.

The broader the platform, the more assumptions the application may inherit.

A solution can therefore be technically capable and still be economically wrong.

Four Costs of a Technical Choice

A practical decision framework starts by separating at least four costs.

1. Implementation Cost

How difficult is it to build ourselves?

2. Adoption Cost

How difficult is it to understand, integrate, and learn the solution?

3. Ownership Cost

How difficult is it to operate, debug, maintain, upgrade, and govern?

4. Exit Cost

How difficult is it to remove the dependency later?

The fourth is particularly important.

Engineers frequently evaluate technology as though today's architecture will remain unchanged indefinitely.

It will not.

Requirements change.

Teams change.

Scale changes.

Technology changes.

Business priorities change.

A technology with a high exit cost deserves stronger evidence before adoption.

This gives us an important concept:

Architectural reversibility is itself a form of value.

A cheap decision that is easy to reverse is fundamentally different from a cheap decision that locks the system into a new model for years.

Technology Gravity

Some technologies create what might be called **technology gravity**.

Once introduced, they attract other technologies and concepts around themselves.

A seemingly small decision becomes an ecosystem:

```
Technology
  ↓
additional components
  ↓
new operational practices
  ↓
new deployment requirements
  ↓
new team knowledge
  ↓
new architecture
```

This is not necessarily a flaw.

Ecosystems exist because integrated capabilities create value.

But the ecosystem should be considered part of the cost.

When evaluating a technology, it is worth asking:

What else becomes likely once we choose this?

A technology should not be evaluated in isolation if adopting it changes the set of technologies and practices that become attractive or necessary afterward.

Complexity Ownership

This leads to a more fundamental question than "build or buy":

Who should own this complexity?

There are several possible answers.

The application can own it.

A library can own the implementation mechanism.

A framework can own part of the application lifecycle.

A platform can own execution.

An external service can own an entire capability.

There is no universal correct answer.

The goal is not maximum ownership.

The goal is **appropriate ownership**.

A useful principle is:

Keep important application behavior where the application can understand, test, debug, and change it.

Delegate commodity mechanisms when doing so removes more complexity than it introduces.

This Changes How I Think About Frameworks

Frameworks sit in an interesting middle ground.

They absolutely impose a model.

Even a code-centric application framework can influence:

- structure,
- lifecycle,
- dependency management,
- execution,
- testing,
- and extension mechanisms.

So a framework should not receive a free pass simply because the configuration is written in code.

But frameworks can still be an excellent choice when their conventions solve recurring engineering problems at a favorable cost.

The relevant distinction is not:

"Frameworks are good; platforms are bad."

It is:

"How much architectural control am I giving away, and is the reduction in complexity worth it?"

A framework that fits naturally into the application may justify its commitment.

A platform that becomes the primary owner of an important workflow may not.

A Better Default for Code-Owned Projects

For projects where I control the source code, architecture, and long-term maintenance, a useful default emerges:

Code owns the architecture. Libraries provide capabilities. Frameworks must justify their programming model. External execution platforms are exceptions, not defaults.

This is not an ideology against tools.

It is a bias toward architectural ownership.

The default path becomes:

Problem

↓

Can a library solve the mechanism?

↓
Can a framework help without taking ownership of the architecture?
↓
Can a thin custom layer express the application-specific behavior?
↓
Only then consider a platform

This bias is particularly useful for engineers who are capable of implementing substantial infrastructure themselves.

Their ability is both an advantage and a trap.

It lowers the cost of building.

But it can also create the temptation to build everything.

Therefore the rule cannot be:

"Build it because I can."

Nor can it be:

"Use it because someone already built it."

The correct question is:

"Which ownership boundary gives the lowest total engineering cost?"

And Then AI Agents Change the Equation

AI agents introduce something fundamentally different into this discussion.

Traditional tools generally delegate **mechanisms** or **execution**.

AI agents can delegate **labor** itself.

A conventional workflow might be:

Human
↓
decides what to do
↓
tool executes

An AI coding agent can instead receive:

```
Goal
Constraints
Acceptance criteria
```

and perform:

```
research
planning
implementation
testing
debugging
refactoring
```

The nature of delegation has changed.

The system is no longer merely executing a procedure we designed.

The agent can participate in designing the procedure.

It may choose what to inspect, which approach to take, which tools to use, and how to revise its implementation after observing the results.

This is why AI agents should not simply be placed at the far right of a "library → framework → platform" spectrum.

They introduce a different dimension.

Delegation of Labor vs. Delegation of Ownership

This distinction is critical.

Labor is the work performed to produce an outcome.

Ownership is responsibility for defining what should be done, under what constraints, what counts as acceptable, and whether the result should ultimately be trusted and adopted.

These are not the same thing.

A developer may delegate implementation labor to an AI agent while retaining ownership:

```
Human
↓
intent
constraints
acceptance criteria
```

↓
Agent
↓
implementation
↓
Human ownership

This is fundamentally different from delegating the application's execution model to an external platform.

In the first case, the agent can produce an artifact that comes back into the developer's repository.

The artifact can be:

- inspected,
- tested,
- reviewed,
- modified,
- refactored,
- rejected,
- or deleted.

The important property is not that AI produced it.

The important property is that **the output can become an owned artifact of the codebase.**

Open Source Tools and AI Agents Can Therefore Differ Fundamentally

Consider two situations.

In the first, you adopt an open-source platform.

You have access to its source code, but your application runs according to its architecture.

In the second, an AI agent studies your repository and produces a tailored implementation for your specific problem.

Both involve open source or reusable technology somewhere in the process.

But their relationship to ownership is different.

In the first:

Your system
↓

```
their runtime
  ↓
their architecture
```

In the second:

```
Your problem
  ↓
Agent
  ↓
tailored implementation
  ↓
your repository
```

The agent may have performed a large amount of labor without becoming the owner of the resulting system.

This suggests a powerful distinction:

Prefer delegation of labor over delegation of ownership when the resulting artifact can remain under your control.

What Labor Should We Delegate?

Not all labor is equally suitable for delegation.

A useful criterion is not simply difficulty.

It is **verifiability**.

Some work has relatively objective acceptance conditions:

```
rename all references
migrate this API
generate tests for these cases
implement this known algorithm
convert this representation
```

This work is highly delegatable.

Other work requires substantial judgment:

```
choose the architecture
interpret an ambiguous requirement
```

decide the business trade-off
determine acceptable risk

These tasks can still benefit from AI, but full delegation is more dangerous because the acceptance criteria themselves may be uncertain.

There is a crucial difference between:

You define the problem.
AI solves it.

and:

AI interprets the problem.
AI defines what "good" means.
AI solves its interpretation.

The second involves far more delegated judgment.

Assistance Is Not the Same as Delegation

There is another mode of human-AI interaction that sits between manual work and full delegation:

collaboration.

Instead of telling the agent:

Implement this.

we can ask it to:

Propose an approach.

Then:

What assumptions are you making?

Then:

What could make this design wrong?

Then:

Give me the strongest argument against your own approach.

Then:

What alternatives did you reject?

This kind of interaction—sometimes described as grilling—does something important.

It lets the AI perform exploratory and analytical labor while the human remains deeply involved in judgment.

The resulting interaction is not simply:

Human → Agent → Answer

but:

Human
↕
Agent
↕
exploration
criticism
refinement

This can be especially valuable before implementation.

Does Grilling Eliminate Review?

No.

It can reduce some forms of uncertainty, but it does not eliminate independent verification.

An AI can generate a flawed assumption and then construct an internally consistent argument around it.

It can also critique its own implementation while preserving the same underlying misconception that produced the implementation.

So:

Self-critique is not equivalent to independent verification.

Grilling improves reasoning quality.

It does not turn the producer into an independent reviewer.

This distinction becomes increasingly important as AI-generated output grows in volume.

Review Is Also Labor

Review is usually treated as something humans must do after AI has finished.

But review itself is labor.

It includes:

```
requirement comparison
consistency checking
bug hunting
edge-case discovery
security analysis
architecture review
behavior validation
```

A large portion of this labor can itself be delegated.

For example:

```
Builder
  ↓
AI reviewer
  ↓
Security reviewer
  ↓
Test generation
  ↓
Static analysis
  ↓
Simulation
  ↓
Human
```

The goal is not to eliminate human review.

The goal is to move humans toward the parts of review that require actual ownership and judgment.

Ownership of Acceptance Is Different

This gives us another important distinction:

The labor of review can be delegated without delegating ownership of acceptance.

An AI system can search for bugs.

Another agent can challenge the architecture.

Automated tests can verify behavior.

Static analysis can find inconsistencies.

Security tools can identify suspicious patterns.

But someone still needs to decide:

Is this result acceptable for this system?

That is closer to ownership than to mechanical labor.

The human may therefore stop reading every line manually while still retaining responsibility for the acceptance boundary.

This is one of the most important consequences of AI-assisted engineering.

The objective is not necessarily to keep humans doing all the work.

It is to keep humans responsible for the decisions that matter.

A New Dimension: Decision Delegation

Traditional delegation can be described as:

Delegation of mechanism
Delegation of execution

AI introduces:

Delegation of labor
Delegation of planning
Delegation of decision-making

These are not equivalent.

We can imagine a rough spectrum:

Human does everything
↓
AI assists
↓
AI performs defined labor
↓
AI plans the labor
↓
AI chooses the procedure
↓
AI makes consequential decisions

The farther we move down this path, the more important governance, verification, reversibility, and explicit constraints become.

This is where AI forces us to revisit a question that software architecture has always contained but rarely made explicit:

Who owns the decision?

The Economics of AI Delegation

AI creates an unusual inversion.

Traditional automation reduces execution cost.

AI can reduce the cost of producing complex artifacts.

But as production becomes cheaper, verification can become the bottleneck.

Suppose a developer used to spend:

3 days coding

Now an agent can generate the implementation in:

2 hours

That sounds like an enormous improvement.

But if the developer then needs two days to understand and verify the generated implementation, the bottleneck has simply moved.

So again:

Complexity did not disappear. It moved from production to verification.

This suggests a new engineering requirement:

We must automate verification as aggressively as we automate production.

A Useful Principle for AI Engineering

One principle emerges naturally:

Delegate generation aggressively; delegate acceptance conservatively.

This does not mean humans should inspect every line of every AI-generated change.

It means that the acceptance boundary should remain explicit.

The more autonomous the generation process becomes, the stronger the evidence supporting acceptance should become.

That evidence can come from:

- tests,
 - static analysis,
 - independent review,
 - formal invariants,
 - reproducible experiments,
 - security checks,
 - architecture constraints,
 - and human judgment.
-

The Deeper Framework

At this point, the original "build or buy" question has expanded into a more general model.

A technical decision can involve at least these dimensions:

Complexity

↓

Where does it live?

Ownership

↓

Who is responsible for it?

Delegation

↓
Who performs the labor?

Mental Model
↓
What way of thinking must the team adopt?

Reversibility
↓
How expensive is exit?

Verification
↓
How do we know the delegated result is correct?

Gravity
↓
What additional commitments does this choice attract?

This gives us a richer way to evaluate technologies.

Instead of asking:

Which option has the most features?

we ask:

Which option gives us the best ownership and complexity distribution for this problem?

A Practical Decision Framework

Before adopting a significant technology, I would now ask:

What complexity does it remove?

Be specific.

"Less code" is not a sufficient answer.

What complexity does it introduce?

Include learning, operations, configuration, debugging, and organizational knowledge.

Who owns the resulting behavior?

The application, the library, the framework, the platform, or an external service?

How deeply must the codebase adopt its mental model?

Can the dependency remain local, or does the entire system have to think in its terms?

What is the exit cost?

Can we replace the implementation, or would we have to replace the architecture?

What is the verification strategy?

Especially when the delegated component produces or changes behavior that matters.

What does this decision attract?

Will this choice create a technology ecosystem or operational gravity around itself?

Can the result remain an owned artifact?

This question becomes especially important with AI.

A Broader Principle

These questions point toward a more general idea:

Every engineering decision is partly a decision about the location, ownership, and movement of complexity.

This explains why apparently unrelated technologies can produce similar engineering experiences.

A platform can move execution complexity outside the application.

A framework can move lifecycle complexity into a programming model.

A functional runtime can move concurrency complexity into a different computational model.

A managed service can move operational complexity to a provider.

An AI agent can move implementation labor to an autonomous system.

None of these are inherently good or bad.

They are different forms of delegation.

And each form creates a different relationship between:

labor
complexity

knowledge
control
ownership
verification

Delegation of Labor vs. Delegation of Ownership

This may ultimately be the most useful distinction.

When we delegate **labor**, someone—or something—does work that contributes to an artifact we still own.

When we delegate **ownership**, we move responsibility for the behavior, execution, or decision itself to another system.

The first can be highly efficient while preserving architectural control.

The second can be equally efficient, but it creates dependency and changes who controls the system.

This leads to a powerful rule:

Delegate labor when possible. Delegate ownership when the economics genuinely justify it.

And when ownership is delegated, do so deliberately rather than accidentally.

The Goal Is Not Maximum Control

It would be easy to turn all of this into an anti-tool philosophy.

That would be another mistake.

Maximum control is not the objective.

Owning a difficult distributed system yourself can be much more expensive than adopting a mature solution.

A highly autonomous platform can be exactly the right architecture when the problem is sufficiently complex, stable, generic, and expensive to reproduce.

Likewise, a sophisticated programming model can be worth its cognitive cost when the problem genuinely requires it.

The objective is not:

"Build everything."

Nor is it:

"Use everything that already exists."

The objective is:

Find the cheapest sustainable location for complexity while preserving the ownership and control that the system actually requires.

From "Build vs. Buy" to "Where Should Complexity Live?"

This is the question I now find more useful than the traditional build-vs-buy debate.

Not:

Should I write this?

Not:

Is there a tool for this?

But:

Where should this complexity live?

And, when AI is involved:

Who should perform the labor, who should make the decision, and who should own the result?

Those questions turn technology selection from a feature comparison into an architectural decision.

They also explain why the apparently cheapest solution can sometimes become the most expensive one.

Because the price of delegation is not only what you pay to adopt something.

It is also:

the complexity you inherit, the mental model you must learn, the control you surrender, the knowledge you accumulate, the verification you need, and the cost of leaving later.

That is the hidden cost of delegation.